Finite Automata

Zhengfeng Ji, *Department of Computer Science and Technology, Tsinghua University*

Outline

DFA, NFA, and ϵ-NFA

1 An Informal Picture

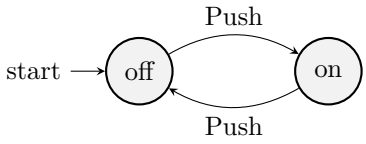
Read Sec 2.1 of the textbook.

External actions drive the changes in the internal state.

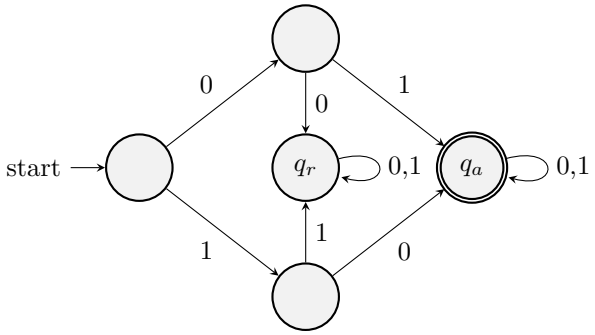
2 What is a Finite Automaton

2.1 Examples

- Finite automata are very simple ***models of computation*** that will be our focus for the next four to five weeks.
- Computation with **finite** memory
- Example 1. A finite ***automaton*** for the light switch.



- Example 2. A finite automaton for accepting all binary strings that have length at least two and whose first two letters are different.



- Example 3. A finite automaton for accepting all binary strings that are 4 (mod 7).

		0	1
$\rightarrow$	0	0	1
	1	2	3
	2	4	5
	3	6	0
$*$	4	1	2
	5	3	4
	6	5	6

- We will talk about deterministic finite automata first.
- Models we will not cover:

  Mealy machines (outputs determined by the input symbol and current state)

  Moore machines (outputs determined by the current state only)

2.2 Formal definition of DFA

Mathematical definition

> 1. A ***finite*** set of states Q
> 2. Input alphabet: A finite set of symbols Σ
> 3. ***Transition function*** $\delta : Q \times \Sigma \rightarrow Q$
> 4. A start state $q_0 \in Q$
> 5. A set of final states $F \subseteq Q$
>
> A deterministic finite automaton (DFA) is a five-tuple $A = (Q, \Sigma, \delta, q_0, F)$

2.3 Transition diagrams

See Examples 1 and 2.

They are best for visualization

2.4 Transition tables

Columns of state transitions

See Example 3 above.

They are machine friendly

2.5 How a DFA processes strings

1. It starts in the start state.
2. It enters a new state according to the transition function δ.
3. It accepts ***if and only if*** the final state is accepting.

- Examples

> **Example**. Design a DFA that accepts all and only the strings of 0's and 1's that have a 01 somewhere. That is, $L = \{w \mid w = x01y\}$.

The automaton needs to remember:

1. Has it seen a 01? If so, it accepts regardless of further input.
2. Has it never seen a 01 but the most recent input is 0?
3. Has it never seen a 0?

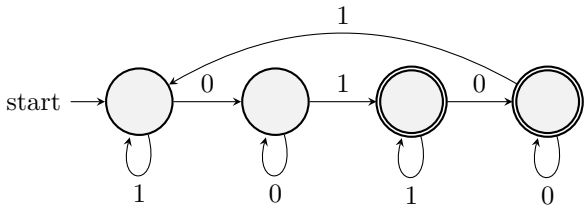
This gives us a simple automaton with three states.

> **Example**. Design a DFA that accepts all and only the strings of 0's and 1's that have 01 as the last two letters. That is, $L = \{w \mid w = x01\}$.

Note the differences at the accepting node.

> **Example**. Design a DFA that accepts all strings over $\{0, 1\}$ in which there is an odd number of occurrences of 01.

For example, 01000 should be accepted while 01001 should not.



2.6 The language of a DFA

- What do we mean by calling a language ***regular***?

  A language is regular if it is the set of strings accepted by a finite automaton.

  Regular: ***arranged in or constituting a constant or definite pattern, especially with the same space between individual instances.***

- Succinct and precise, and leaves no ambiguities

2.7 Extended transition function

- How to define $\hat{\delta}(q, w)$ for a string w?

  Write $w = xa$ and inductively define $\hat{\delta}$.

  We can write the language of the DFA using the extended transition function: $L(A) = \left\{w \mid \hat{\delta}(q_0, w) \in F\right\}$.

2.8 Are regular languages countable or not?

- Yes!
- This means that there are languages $L \subseteq \{0, 1\}^*$ that are not regular.

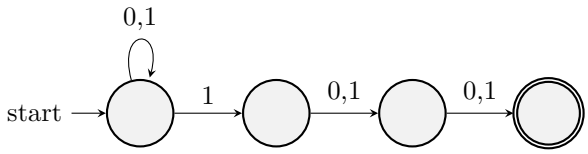
3 Nondeterministic Finite Automata

3.1 Nondeterminism

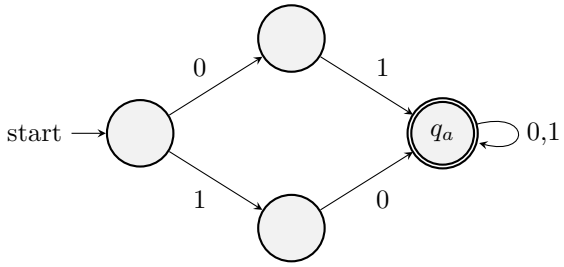
The machine can make ***guesses*** (non-deterministic choices) during computation.

3.2 Example

- Binary strings whose 3rd letter from the end is a 1.



- Example 2 with NFA:



3.3 Dead states and DFAs missing some transitions

They are not DFAs by definition, but they are in fact NFAs.

See Example 2.

3.4 Formal definition

It is still a five-tuple, but now the transition function is different. (***The only difference***)

Nondeterminism is reflected in the fact that $\delta(q, a)$ is a set of states representing all possible target states when the current state is q, and the input symbol is a.

3.5 Extended transition function

- Write $w = xa$ and take the union of $\delta(p_i, a)$ for all $p_i \in \hat{\delta}(p_0, x)$.

3.6 The language of an NFA

- Intuition and formal definition:

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}.$$

The string is in the language as long as ***there exists*** a nondeterministic choice of path leading to acceptance.

4 DFA and NFA Equivalence

4.1 What does it mean?

> Any language L has a DFA if and only if it has an NFA.

The direction from DFA to NFA is easy as any DFA is a special NFA.

4.2 NFA to DFA: idea

Subset Construction

- The transition in an NFA is to a ***set of states*** (representing the nondeterministic choices).
- Can we use a set of states (of the NFA) as ***the state*** of a DFA?
- Transition: Union of target states for each state in the set.

4.3 NFA to DFA: proof

Proof. Let N be an NFA. We design a DFA D such that $L(D) = L(N)$.

The five components of D are:

1. The set of states Q_D is the power set 2^{Q_N}.
2. The alphabet is the same Σ.
3. The transition rule δ_D is

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a).$$

4. Start state $\{q_0\}$.
5. Final states: any set that has nontrivial overlap with F.

Now we need to show that the construction works, that is, $w \in L(N)$ if and only if $w \in L(D)$.

$w \in L(N)$ means $\delta_N(q_0, w)$ contains states in F and $w \in L(D)$ means $\delta_D(\{q_0\}, w)$ contains states in F.

Induction on the length $|w|$ and prove that $\delta_N(q_0, w) = \delta_D(\{q_0\}, w)$.

See Theorem 2.11 of the textbook.

□

4.4 Comments

DFA simulation of an NFA could have an exponentially large number of states.

Example: all bit strings whose n-th symbol from the end is 1.

Show that any DFA accepting those strings will have at least 2^n states. (Pigeonhole principle)

Yet there is an NFA with $n + 1$ states for this. (The NFA first loops for a while on $0, 1$ and then moves n steps sequentially).